COM431

Server Side Development

Week 5

Laravel – Factories, Seeders and Relationships

Peter Nicholl

Simon Fraser

Last week

OOP and NameSpaces

Laravel: Models, Migrations and Eloquent

Mini challenge using Tinker and TablePlus

Creating the dB structure – without seeding data

Name notes Primary id Search fo

#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	EMPTY
2	user_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	users(id)	EMPTY
3	title	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	EMPTY
4	body	longtext	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	EMPTY
5	image_path	varchar(255)	utf8mb4	utf8mb4_unicode_ci	YES	NULL	EMPTY	EMPTY	EMPTY
6	time_to_read	smallint	NULL	NULL	NO	2	EMPTY	EMPTY	EMPTY
7	is_published	tinyint(1)	NULL	NULL	NO	1	EMPTY	EMPTY	EMPTY
8	priority	smallint	NULL	NULL	NO	1	EMPTY	EMPTY	EMPTY
9	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	EMPTY
10	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	EMPTY
11	deleted_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	EMPTY

php artisan migrate

```
~/S/mynotes-app  git main ?2  sail artisan migrate

 INFO  Running migrations.

2023_03_03_150141_create_notes_table ..................... 74ms DONE
```

```
~/S/mynotes-app  git main ?2  sail artisan migrate:rollback

 INFO  Rolling back migrations.

2023_03_03_150141_create_notes_table ..................... 28ms DONE
```

- notes
  - columns
    - id bigint
    - user_id bigint
    - title varchar(255)
    - body longtext(4294967295)
    - image_path varchar(255)
    - time_to_read smallint
    - is_published tinyint
    - priority smallint
    - created_at timestamp
    - updated_at timestamp

Factories and Seeders

- Factories are blueprints

```
public function definition(): array
{
return [
'name' => fake()->name(),
'email' => fake()->unique()->safeEmail(),
'email_verified_at' => now(),
'password' => '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', // password
'remember_token' => Str::random(10),
];
}
```

- Seeders add the data

```
User::create([
'name' => "Peter Nicholl",
'email' => 'pete@abc.com',
'password' => Hash::make('qwerty1234'),
'email_verified_at' => now(),
]);
User::factory()->count(2)->create();
```

Creating Factories

Typical approach to make a Factory:

php artisan make:factory PhotoFactory

php artisan make:factory PostFactory

php artisan make:factory NoteFactory

php artisan make:factory TagFactory

- migration
- resource controller
- 4 factory
- seeder
- form request
- policy

```php
<?php
namespace Database\Factories;
use Illuminate\Database\Eloquent\Factories\Factory;
/**

* @extends \Illuminate\Database\Eloquent\Factories\Factory<\App\Models\Model>

*/

class NoteFactory extends Factory

{

/**

* Define the model's default state.

*

* @return array<string, mixed>

*/

public function definition(): array

{

return [

    'user_id' => 1,  // make sure this user exists

    'title' => $this->faker->unique()->sentence(),

    'body' => $this->faker->text(),

    'image_path' => $this->faker->imageUrl(640, 480),

    'is_published' => 1,

    'time_to_read' => $this->faker->numberBetween(1, 10),

    'priority' => $this->faker->numberBetween(1,5)

    ];

 }

}
```

https://github.com/fzaninotto/Faker

Relationships

Eloquent Relationships

- posts table = likes table
- students table = subjects table
- users table = notes table

- One to one
- One to many
- One of many
- Many to many
- Custom types

app > Models > User.php > ...

```php
<?php

namespace App\Models;

use App\Models\Note;
use Laravel\Sanctum\HasApiTokens;
use Illuminate\Notifications\Notifiable;
use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable implements MustVerifyEmail
{
    use HasApiTokens, HasFactory, Notifiable;

    /**…
    protected $fillable = […
    ];

    /**…
    protected $hidden = […
    ];

    /**…
    protected $casts = […
    ];

    public function notes(){
        return $this->hasMany(Note::class);
    }
}
```

app > Models > Note.php > ...

```php
<?php

namespace App\Models;

use App\Models\User;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Note extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'body',
        'image_path',
        'time_to_read',
        'is_published',
        'priority',
    ];

    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

Our user model has a **one to many** relationship with the notes model.

- User has many notes
- Note belongs to a user

```php
if(!$note->user->is(Auth::user())) {
    return abort(403);
}
```

Visualising the Relationship

MySQL 8.0.32 : TLSv1.2 : myNotes : mynotes_app : notes

Items | Queries | History

Search for item...

Tables

- fai
- mi
- no
- pa
- pe
- us

notes | users

id	user_id	title	body	image_path	time_to_read	is_published	priority	created_at	updated_at
1	1	Yes, it's...	Taken a very v...	https://computing.ulster...	4	1	5	2023-03-05 11:49:32	2023-03-05 11:49:32
2	1	Autem ac...	Vitae quia ani...	https://via.placeholder.c...	5	1	2	2023-03-05 11:49:32	2023-03-05 11:49:32
3	1	Laborum...	Explicabo et te...	https://via.placeholder.c...	8	1	2	2023-03-05 11:49:32	2023-03-05 11:49:32
4	1	Sit repell...	Quos quod eu...	https://via.placeholder.c...	4	1	5	2023-03-05 11:49:32	2023-03-05 11:49:32
5	1	Quo repel...	Ab illum sunt s...	https://via.placeholder.c...	6	1	4	2023-03-05 11:49:32	2023-03-05 11:49:32
6	1	Consequ...	Et perferendis l...	https://via.placeholder.c...	7	1	5	2023-03-05 11:49:32	2023-03-05 11:49:32
7	1	Delectus...	Assumenda eo...	https://via.placeholder.c...	2	1	4	2023-03-05 11:49:32	2023-03-05 11:49:32
8	1	Harum eo...	Qui iusto nequ...	https://via.placeholder.c...	6	1	2	2023-03-05 11:49:32	2023-03-05 11:49:32
9	1	Quod inci...	Incidunt perfer...	https://via.placeholder.c...	3	1	4	2023-03-05 11:49:32	2023-03-05 11:49:32
10	1	Aperiam li...	Nobis voluptat...	https://via.placeholder.c...	9	1	5	2023-03-05 11:49:32	2023-03-05 11:49:32
11	2	Made by...	Made by me A...	https://via.placeholder.c...					

- Open in new tab
- Open structure
- Item overview
- Copy name
- Pin to top
- Export...
- Import
- New
- Copy Script As
- Clone...
- Truncate...
- Delete...
- Generate diagram
  - Empty Diagram
  - Generate Diagram
  - Generate Diagram selected Items

```php
Schema::create('notes', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
```

notes

id	bigint unsigned
user_id	bigint unsigned
title	varchar(255)
body	longtext
image_path	varchar(255)
time_to_read	smallint
is_published	tinyint(1)
priority	smallint
created_at	timestamp
updated_at	timestamp

users

name	type
id	bigint unsigned
name	varchar(255)
email	varchar(255)
email_verified_at	timestamp
password	varchar(255)
remember_token	varchar(100)
created_at	timestamp
updated_at	timestamp

Add

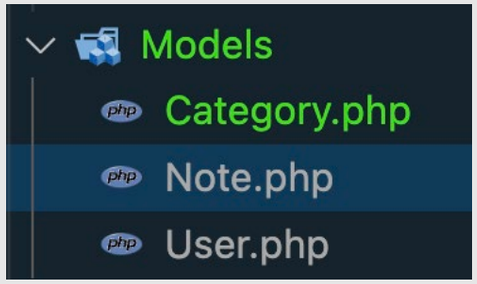
Plugin Manager

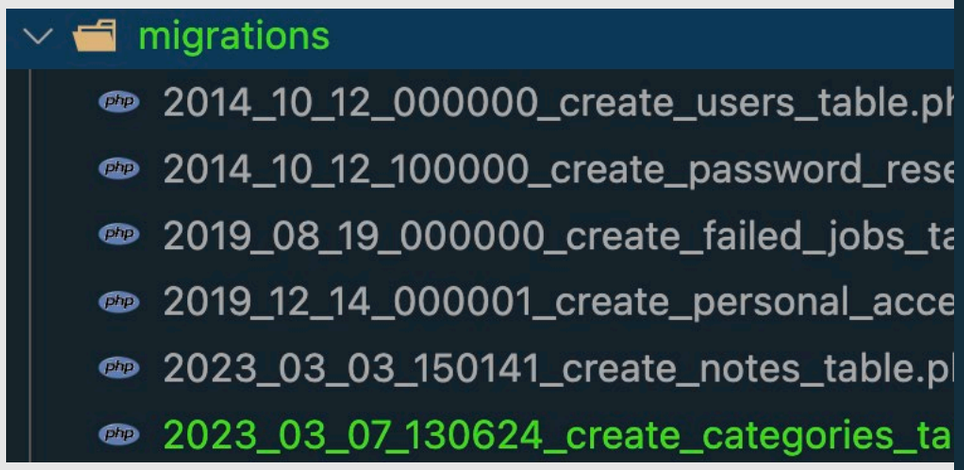
Installed Plugins

Diagram Generator (4.0)
TablePlus
This plugin helps you generate database diagram

Many to Many Relationship

php artisan make:model Category -m






```
$table->string('topic')->unique(); // This will allow for a
slug_url
```

2023_03_07_130624_create_categories_table.php U

database > migrations > 2023_03_07_130624_create_categories_table.php > ...

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->string('topic')->unique();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('categories');
    }
};
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use App\Models\Flight;

class Client extends Model
{
use HasFactory;

public function flights()
{
return $this->belongsToMany(Flight::class, 'tickets', 'client', 'flight')
->as('ticket')
->withTimestamps()
->withPivot('seat');
}
}
```

- Models
  - Client.php
  - Flight.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use App\Models\Client;

class Flight extends Model
{
use HasFactory;

public function clients()
{
return $this->belongsToMany(Client::class, 'tickets', 'flight', 'client')
->as('ticket')
->withTimestamps()
->withPivot('seat');
}
}
```

- migrations
  - 0001_01_01_000000_create_users_table.php
  - 0001_01_01_000001_create_cache_table.php
  - 0001_01_01_000002_create_jobs_table.php
  - 2024_04_10_180541_create_clients_table.php
  - 2024_04_10_180645_create_flights_table.php
  - 2024_04_10_180806_client_flight.php

```php
public function up(): void
{
Schema::create('tickets', function (Blueprint $table) {
$table->foreignId('client');
$table->foreignId('flight');
$table->primary(['client', 'flight']);
$table->string('seat');
$table->timestamps();
});
}
```

```php
public function up(): void
{
Schema::create('clients', function (Blueprint $table) {
$table->id();
$table->string('name');
$table->timestamps();
});
}
```

```php
public function up(): void
{
Schema::create('flights', function (Blueprint $table) {
$table->id();
$table->string('name');
$table->string('airline');
$table->timestamps();
});
}
```

Creating a resourceful controller with a model for Category

```
5   use App\Models\Category;
6   use Illuminate\Http\Request;
```

app > Http > Controllers > CategoriesController.php > ...

```
8   class CategoriesController extends Controller
34      /**
35       * Display the specified resource.
36       */
37      public function show(Category $category)
38      {
39          //
40      }

        /**
         * Show the form for editing the specified resource.
         */
        public function edit(Category $category)
        {
            //
        }

        /**
         * Update the specified resource in storage.
         */
        public function update(Request $request, Category $category)
        {
        }
```

php artisan make:controller

```
~/S/l11-mynotes-app  git main !3  sail artisan make:controller

What should the controller be named?
CategoriesController

Which type of controller would you like?
Resource

What model should this resource controller be for? (Optional)
Category

A App\Models\Category model does not exist. Do you want to generat…
Yes

INFO  Model [app/Models/Category.php] created successfully.

INFO  Controller [app/Http/Controllers/CategoriesController.php] created
successfully.
```

web.php | Category.php U

app > Models > Category.php > ...

```
1   <?php
2
3   namespace App\Models;
4
5   use Illuminate\Database\Eloquent\Factories\HasFactor
6   use Illuminate\Database\Eloquent\Model;
7
8   class Category extends Model
9   {
10      use HasFactory;
11      protected $fillable = ['topic'];
12  }
```

php artisan make:migration create_category_table

migrations

- 0001_01_01_000000_create_users_table.php
- 0001_01_01_000001_create_cache_table.php
- 0001_01_01_000002_create_jobs_table.php
- 2024_03_20_111106_create_notes_table.php
- 2024_03_24_131622_create_category_table.php

Migrations

Dealing with Foreign keys in the relationship

- Using onDelete() option to delete related links

cascade or null to not delete

(pre-Laravel 9)

2022_03_02_225607_create_events_table.php

```php
 * @return void
 */
public function up()
{
    Schema::create('events', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('user_id');
        $table->unsignedBigInteger('person_id');
        $table->string('name');
        $table->date('date');
        $table->string('repeat_every');
        $table->string('notify_days_before');
        $table->timestamps();

        $table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');
        $table->foreign('person_id')->references('id')->on('people')->onDelete('cascade');
    });
}
```

2022_03_02_225607_create_events_table.php

```php
 * @return void
 */
public function up()
{
    Schema::create('events', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->foreignId('person_id')->constrained()->onDelete('cascade');
        $table->string('name');
        $table->date('date');
        $table->string('repeat_every');
        $table->string('notify_days_before');
        $table->timestamps();
    });
}
```

Many to Many Relationship

php artisan make:migration create_category_note_table

Most important is to include a way to link the tables categories and notes

database > migrations > 2023_03_07_132915_create_category_note_table.php > ...

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('category_note', function (Blueprint $table) {
            $table->foreignId('category_id')->constrained()->onDelete('cascade');
            $table->foreignId('note_id')->constrained()->onDelete('cascade');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('category_note');
    }
};
```

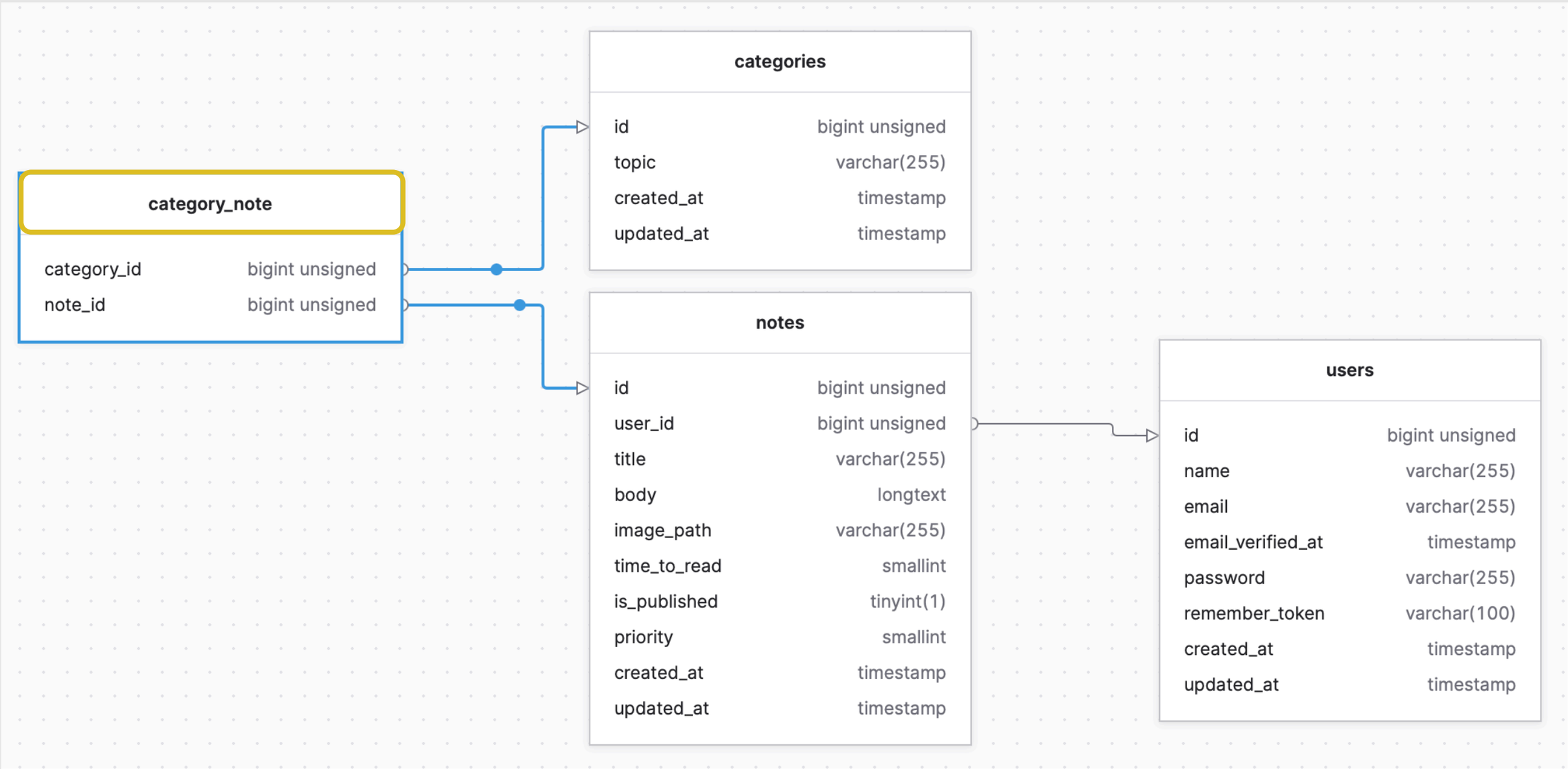
php artisan migrate

```
~/S/mynotes-app  git main ?3  sail artisan migrate

INFO  Running migrations.

2023_03_07_130624_create_categories_table ........... 90ms DONE
2023_03_07_132915_create_category_note_table ....... 63ms DONE
```

Laravel Eloquent 'magic' happens when you use the alphabetic table order and the singular model names

Many to Many Relationship

TablePlus Diagram



Eager Loading (to reduce N+1 queries)

HTML

Notes that use this Category: "Made by me"

1 week ago

jQuery

Notes that use this Category: "Delectus aut illo velit aut ut."

1 week ago

Laravel

Messages | Timeline | Exceptions | Views 15 | Route | Queries 3 | Models 15 | Gate | Session | Request | GET categories | 2MB | 52.94ms | 8.2.3

3 statements were executed — 2.72ms

```
select * from `users` where `id` = 2 limit 1
```
1.95ms /vendor/laravel/framework/src/Illuminate/Auth/EloquentUserProvider.php:59 mynotes_app

```
select * from `categories` order by `topic` asc
```
330µs /app/Http/Controllers/CategoriesController.php:17 mynotes_app

```
select `notes`.*, `category_note`.`category_id` as `pivot_category_id`, `category_note`.`note_id` as `pivot_note_id`
from `notes` inner join `category_note` on `notes`.`id` = `category_note`.`note_id` where
`category_note`.`category_id` in (1, 2, 3, 4, 5, 6, 9) and `notes`.`deleted_at` is null
```
440µs /app/Http/Controllers/CategoriesController.php:17 mynotes_app

```
public function index()
{
$categories = Category::orderBy("topic", "asc")->with('notes')->get();

return view('categories.index', compact('categories'));
}
```

Lab Class – Using Factories, Seeders and Relationships

Based on the Lecture Slides and Support Material

You may watch the following videos for step-by-step demo

https://www.youtube.com/watch?v=9O_WD5zQGxM&t=600s

In the tenth episode of the "30 Days to Learn Laravel" series we use factories to allow us to scaffold example data for the purposes of both testing and preparing a local environment with information on the Jobs.

https://laracasts.com/series/30-days-to-learn-laravel-11/episodes/11

In the eleventh episode of the "30 Days to Learn Laravel" focuses exclusively on two key Eloquent relationship types: belongsTo() and hasMany()

https://www.youtube.com/watch?v=x1UCiE0hZiw&t=293s

In Episode 12 of the "30 Days to Learn Laravel 12" focuses on Pivot Tables and BelongsToMany Relationships

- Use GIT to prepare your Jobs project (1)
- Use the Model Job and add a Factory (2)
- Use a Seeder to initialize the JobListing Table (3)
- Mini Challenges to create a new Laravel Project with a person that has many books and sets up 10 users, each with 5 books (4)